

EngiSync — Product Specification & Architecture Document

The Engineering Collaboration & Project Intelligence Platform

Version 1.0 — maintained alongside the codebase

27 July 2026

1. Vision & Product Philosophy
2. Users & Personas
3. UX & Information Architecture
 - 3.1 Navigation □
 - 3.2 The command palette □
 - 3.3 Dashboard philosophy □
 - 3.4 Design language □
4. The Engineering Project Lifecycle □
5. Projects & Workspaces
6. Security Architecture & Zero Trust
 - 6.1 The policy layer □
 - 6.2 Request validation chain □
 - 6.3 Platform hardening □
 - 6.4 Account lifecycle □
7. Collaboration
8. Tasks, Notifications & Reminders
9. Finance & Contribution Module □
10. Repository & Publication □
11. Group Membership Lifecycle □
12. Leader & Supervisor Experience □
13. AI Integration
14. Mobile & Responsiveness □
15. Performance & Scalability
16. Future SaaS & Multi-Tenancy □
17. Phased Roadmap
18. Decision Log — Rejected & Replaced

1. Vision & Product Philosophy

EngiSync is an **Engineering Collaboration & Project Intelligence Platform** for university engineering and computer science education. It guides student project groups from first idea to published, archived work, while giving supervisors the visibility to oversee many groups without drowning in them, and giving departments a knowledge base that compounds: every completed project makes the platform more valuable for the next cohort.

Three principles govern every design decision, and they are enforced in code, not just stated here:

Organised around goals, not features. Students think “I want to know if we’re behind schedule,” not “where is the analytics module.” The platform is structured project-first: one central noun (the project), with everything — tasks, money, documents, meetings, publication — living inside it, and a single personal lens (*My Work*) across all of it.

Humans decide; the system informs. No score publishes a project, no AI verifies a payment, no formula grades a student. Deterministic checks flag; people with names sign off. Every gate in the platform terminates in an accountable human decision, and every such decision is recorded.

Trust is structural, not promised. Financial history is append-only. Published archives have no foreign keys to mutable data. Authorization flows through one tested policy function. Email addresses never render in shared UI. These are properties of the schema and the code, which is the only place trust properties mean anything.

Status convention used throughout this document: ☐ Implemented · ☐ Decided (designed and agreed, not yet built) · ☐ Deferred (consciously postponed, seam prepared) · ☐ Rejected (considered and declined, with reasons — see §18).

2. Users & Personas

The Student (member). Fourth-year engineering student, often on a phone with intermittent connectivity. Needs to know *what to do right now*, prove their contribution, and pay their share of project costs without being defrauded or accused. Everything on their dashboard answers “what needs me today.”

The Group Leader. A student with project-manager responsibilities but no training for them. Their real job is unblocking people: approving joins, verifying payments, assigning unowned work, deciding withdrawals. The leader workspace (§12) leads with queues, not charts.

The Supervisor / Lecturer. Oversees dozens of groups. Needs to know instantly which are behind, who is inactive, and what awaits their signature. Deliberately **read-only** inside projects — with exactly two signing powers: publication approval and withdrawal confirmation. Observing must never blur into doing the students’ work.

The Department Administrator. Manages departments, announcements, the resource hub, and membership. Operates at department level, never inside a group’s private work.

Roles are implemented as: system roles (`GUEST / INDIVIDUAL / ADMIN`), department roles (`ADMIN / SUPERVISOR / MEMBER`), and workspace roles (`LEADER / MEMBER / VIEWER`) plus delegated capabilities (§6). □

3. UX & Information Architecture

3.1 Navigation □

Primary navigation is five items, permanently: **Home** (what needs me now), **My Work** (my tasks/meetings/deadlines across every project), **Projects** (the containers everything lives in), **Repository** (published past work), **Library** (department resources). Everything else — cross-project rollups, role-specific tools, settings — lives in a visually subordinate “More” group, the mobile drawer, and the ⌘K command palette.

The governing rule is **one axis**: the app is organised by container (project-first, the GitHub model), with exactly one personal lens on top. The historical failure mode this replaces was sixteen top-level items where eight were feature-first rollups competing with the same features inside each project — users could not form a mental model because no view was authoritative.

3.2 The command palette □

⌘K / Ctrl+K opens a searchable list of every destination, with keyboard navigation. This is what makes the light sidebar safe: nothing is hidden, it is just not all on screen. It is also the designated seam for future “AI as navigation” — intent matching can be layered onto the palette without changing the interaction.

3.3 Dashboard philosophy □

The dashboard answers exactly one question: **“What should I do right now?”** Three blocks in priority order: *Today* (overdue tasks, today’s meetings with countdown), *Your projects* (sorted behind-schedule-first, each showing stage, pace badge, and how many tasks need you), *Recent activity*. There are deliberately no charts and no counters on the dashboard — counters are not decisions. Statistics live in project insights where “how are we doing overall” is actually the question being asked.

3.4 Design language □

Tailwind + shadcn-style components, dark/light themes, 44px+ touch targets, `env(safe-area-inset-bottom)` on the mobile bottom bar, reduced-motion support, aria-live on forms, empty states that say what to do next. Fixed overlays render through React portals (a `backdrop-filter` ancestor creates a containing block that otherwise traps them — a bug found once and now a standing pattern).

4. The Engineering Project Lifecycle

Every project carries a stage: **Idea → Proposal → Planning → Design → Simulation → Implementation → Testing → Documentation → Presentation → Completed**, grouped for display into four phases (Define / Design / Build / Deliver) so the stepper survives a 360px screen.

Two hard rules:

Stages are advisory, never gates. A team in Planning can still upload documents and hold meetings. Blocking actions by stage would be infuriating and pedagogically wrong — real projects loop back (Testing fails → Design), and the stepper permits movement in both directions so the record reflects what actually happened.

“Are we behind?” is arithmetic, not AI. `stageEnteredAt` plus an optional target end date drive the pacing engine (`lib/lifecycle.ts`): expected progress (elapsed calendar fraction) versus actual progress (stage index), plus independent stall detection (≥ 14 days in one stage = watch, ≥ 21 = behind, regardless of a comfortable deadline). Output is a status — on-track / watch / behind — and a plain-language sentence, identical for every viewer, computed in microseconds, covered by 31 unit tests including the stall-beats-comfortable-deadline case.

5. Projects & Workspaces

A project is the unit of everything: members, stage, tasks, meetings, documents, discussions, quizzes, finance, analytics, publication. 

Known debt, managed: the codebase currently exposes one database entity (`Workspace`) through two route trees (`/workspaces/[id]` for people/access, `/projects/[id]` for plan/milestones). This is the single largest remaining IA flaw. Unification was deliberately sequenced after the feature work at the owner’s direction; the cost is contained by `src/lib/routes.ts` — every link in the application is built through canonical helpers (`projectHome()`, `projectPlan()`, `projectTeam()` ...), so unification is an edit to one file plus redirects, not a hunt through forty pages. 

Group entry is controlled: join codes, optional PINs (bcrypt-hashed), QR invites, member caps, and approval-required mode with a join-request queue.  Group *exit* is a workflow, not a button (§11). 

6. Security Architecture & Zero Trust

6.1 The policy layer

Every permission decision in the application flows through one pure function:

`can(context, action)` — where context is `{role, capabilities, isSystemAdmin, isSupervisor}` and action is one of ~30 named operations (`project.stage.set`, `budget.manage`, `publication.approve` ...). Because `can()` performs no I/O, the entire permission matrix is unit-tested exhaustively — every role against every action, every capability combination, plus

explicit privilege-escalation guards (150+ assertions). Server actions call `authorize(workspaceId, userId, action)` and return its human-readable denial on failure. No server action hand-rolls a role check.

Key rules encoded in the matrix:

- **Leaders** can do everything in their project **except approve their own publication** — separation of duties is a matrix rule, not a UI convention.
- **Supervisors** hold read access to every project in their department plus exactly two writes: `publication.approve` and withdrawal confirmation. A stale capability grant on a downgraded member confers nothing.
- **Capabilities** (`canApprove` , `canManageBudget` , `canInvite`) are leader-delegated per member. “Assistant leader” is a member with `canApprove` — a grant, not a fourth role tier.
- Non-members are denied everything; platform admins bypass workspace roles (first rule in the function, impossible to miss in review).

6.2 Request validation chain □

Identity (Auth.js v5 JWT session) → membership/department standing (`getContext` , one query) → action permission (`can`) → resource ownership (actions verify child records belong to the claimed workspace — e.g. publication file selection is checked against the workspace before archiving, closing the cross-tenant file exfiltration path).

6.3 Platform hardening □

CSP, HSTS (2y, preload), X-Frame-Options DENY, Permissions-Policy, `no-store` on API responses; bcrypt passwords, TOTP 2FA, email verification with real failure surfacing, password reset; rate limiting on auth and financial endpoints; Zod validation on inputs; Prisma parameterised queries (no raw SQL in request paths); file upload size caps; audit log (security-shaped, distinct from the user-facing activity feed by design); PII rule — **email addresses never render in shared UI** — enforced through the single `displayName()` accessor.

6.4 Account lifecycle □

Deletion requires password re-entry (+ TOTP if enabled) and an emailed confirmation token; is blocked for sole leaders of active groups; anonymises shared academic records (“Deleted user”) while hard-deleting personal data; and reserves the email’s SHA-256 hash for 30 days to prevent impersonation-by-re-registration.

7. Collaboration

Strategic decision: integrate, don’t build. □ Live collaborative editing, whiteboards, CAD preview, and typing indicators were evaluated and rejected: each is a multi-year product (operational transforms alone would dominate the roadmap), and serverless hosting cannot hold the websockets they require. A student on intermittent connectivity is better served by excellent asynchronous collaboration than a degraded real-time imitation. External tools

(GitHub, shared documents, simulation software) are linked and referenced, not reimplemented.

What excellent async means here, all ☐: threaded discussions per project; structured documentation with sections, supervisor review, locking, approvals and versioning; supervisor feedback attached to the project (not scattered across WhatsApp); decision-visible workflows (every approval names its approver); and the **activity feed**.

The activity feed is a hybrid by design: events with no row of their own (stage changes, approvals, permission grants, budget events) are stored in an `Activity` table with denormalised actor names (survives account deletion); events that already have rows (tasks completed, files uploaded, meetings scheduled, posts) are **derived at read time** — no write path, no backfill, and they can never drift from the records they describe. The feed showed real history from the moment it deployed.

8. Tasks, Notifications & Reminders

Tasks: priorities, deadlines, assignment, dependencies, recurrence, time logging with live timer, personal + project scope. ☐ The *My Work* page splits an individual's tasks into Overdue / Due today / Coming up, with a per-project breakdown. ☐ Unowned work surfaces on the leader's manage page — “unassigned is how deadlines get missed” is a first-class queue. ☐

Notifications: in-app centre, web push, opt-in email; due-soon generation on dashboard load; every significant workflow event (join request, payment declared/verified/rejected, withdrawal steps, publication steps, permission grants) notifies exactly the people who can act on it, with a deep link. ☐ Meaningful-notification principle: a notification should encourage a specific action, not announce that something exists.

Scheduled/background reminder jobs (cron-driven “meeting in 15 minutes”) are deferred pending a queue (\$15). ☐

9. Finance & Contribution Module ☐

EngiSync is a financial collaboration and verification layer, never a payment processor. Members pay through their own EcoCash/OneMoney/InnBucks/ZIPIT/bank apps; the platform records what was promised, declared, confirmed and spent. This is a legal position as much as an architectural one: holding third-party funds would constitute money transmission under RBZ supervision. The platform never asks for PINs, passwords or OTPs — stated in the UI at the point of payment, where social engineering actually happens, and there is no field in which such a value could be stored.

Mechanics:

- **Contribution requests** (“PCB manufacturing — \$120, \$15 each, due 15 Aug”) with per-member progress bars; payment instruction pages (leader's EcoCash/bank details).
- **Declare → Verify**: members declare payments (method, amount, reference, optional receipt); a leader or `canApprove` member verifies against their own records. There is

deliberately **no “AI checked” status** — checks are advisory flags on the record, so the module functions fully with AI disabled.

- **Deterministic anti-fraud:** transaction reference uniqueness per project **enforced by a database constraint** (the strongest control in the module — a reference cannot be claimed twice, ever, including under race); SHA-256 receipt hashing for exact-duplicate detection; reference format validation per method (warn, never block — providers change formats); paid-at sanity checks; amount-vs-expected-share comparison. Flag wording is tested to never contain accusatory language: the system states facts, it does not allege fraud.
- **Append-only ledger:** every status transition is an immutable `LedgerEvent`. A leader cannot quietly flip a verified payment; the history *is* the audit log. Rejection requires a written reason; the member may **dispute**, which flags the supervisor. The common real dispute — “I paid and the leader says I didn’t” — has a recorded, reviewable trail.
- **Money correctness:** all arithmetic in integer cents (61 unit tests, including the floating-point trap and negative-mirror rounding); per-transaction currency (USD/ZWG) with the exchange rate captured at declaration so historical totals never shift; **only verified money counts** toward collected totals, stated in the UI.
- Expenses with categories, vendors, receipts; full spending visible to every member — group-wide transparency is the real fraud control.

Future payment-provider APIs (EcoCash, ZIPIT) slot in as a new verification path producing `VERIFIED` ledger events; the abstraction requires no redesign. □

10. Repository & Publication □

The department’s permanent knowledge base — the feature whose value compounds, and the answer to “repeated projects because previous work is unavailable.”

Publication gate = artifact checklist + supervisor signature. Universal requirements are few and non-negotiable: abstract (≥ 200 chars), ≥ 3 keywords, license, final report. Code/CAD/BOM/simulation are *recommended*, weighed by the supervisor per project kind — demanding a BOM from a pure-software project is checklist theatre. There is **no numeric score threshold** □: formulas get gamed and block legitimate work; humans publish, scores advise. The submitting leader can never self-approve (§6.1).

Records are snapshots. On approval, file bytes are copied into archive-owned rows; the record holds no foreign keys to users, workspaces or departments — authors are stored as names. The source project can be deleted, authors can delete their accounts, and the published record survives untouched. Bytes copy only at approval, so rejected submissions cost no storage.

Identity & citation: permanent slug (ES-2026-0042), stable URL, generated citation line, download counting, license display.

Search: Postgres `ILIKE` across title/abstract plus array facets (keywords, components, languages, disciplines) — a search for “ESP32” matches whether it appears in prose or only as a component tag. Facet chips are themselves search links. “Related projects” is deterministic

tag overlap, honestly labelled. At departmental scale this is instant; `tsvector` full-text is the upgrade path at tens of thousands of records, not a day-one need. □

Visibility per record (`DEPARTMENT` / `UNIVERSITY`) is the multi-tenancy seam (§16).

11. Group Membership Lifecycle □

Members cannot simply disappear. Leaving is: **request with reason** → **open-task and sole-leader blockers** → **leader decision** → **supervisor confirmation** (when the group belongs to a department; leader approval completes it otherwise) → **removal, record retained**.

Blockers are re-checked at decision time, not just request time — tasks assigned after the request cannot be abandoned through an already-open request. The requester can cancel at any point before completion (“I’m staying” is one click). Rejections at either level require a written reason the member sees. The completed record — who left, when, why, who approved — survives the departure; it is precisely what a supervisor needs when grading individual contribution.

12. Leader & Supervisor Experience □

Leader workspace (`/manage` , accessible to leaders and `canApprove` delegates): leads with the four queues waiting on the leader — join requests, withdrawal requests, payments to verify, unassigned tasks — with a headline count (“5 things waiting on you”). Health tiles (overdue, inactive members, verified funds, days-in-stage) sit below the fold as links; a behind-schedule banner points at what to change. Built entirely from components shared with the queues’ home pages, so behaviour cannot diverge.

Supervisor experience: a portfolio list of supervised projects; per-project read-only detail with stats, milestone approval, structured documentation review (locking, versioned report reviews), feedback that persists on the project; per-student analytics report (contribution %, tasks, attendance, productivity) exportable as PDF; the publication approval queue surfaced at the top of the Repository; withdrawal confirmations surfaced on the supervised-project page. Signals shown to staff are raw and side-by-side — the platform never compresses a student into a single number (§18).

13. AI Integration

AI is a **provider-agnostic enhancement layer** (Gemini/OpenAI/Anthropic/local behind one interface), currently disabled platform-wide by the `AI_DISABLED` kill switch at the owner’s instruction; every AI feature degrades to a clear “out of service” rather than an error. □

Three operating rules, learned from a real incident (auto-triggered recommendations silently exhausting the free-tier quota):

1. **Never on page load.** AI output is requested by a person, never a side effect of navigation. □

2. **Cache and version everything.** An analysis is computed once per input version and stored; re-viewing costs nothing. `cache` (evaluation reports) / `cache` (as a general contract)
3. **Queue anything automatic,** with per-user budgets. `queue` (requires the job queue, \$15)

Where AI legitimately adds value when enabled: meeting summaries and action items, task generation from objectives, document summarisation, evaluation narratives, resource recommendations (opt-in), OCR field extraction on receipts (pre-fill only — a mismatch warns a human; OCR never judges authenticity).

Hard boundaries (\$18 for reasoning): no plagiarism detection, no screenshot forgery verdicts, no calculation “verification,” no auto-approval of anything, no per-student scoring. AI drafts and flags; it holds no authority.

14. Mobile & Responsiveness `mobile`

Mobile is a first-class layout, not a shrunk desktop: bottom navigation (Home / My Work / Projects / Library) with safe-area insets; drawer for everything else; tables become card stacks at small widths (supervisor reports, member lists); the calendar’s month grid becomes an agenda list; the ten-stage stepper collapses to four phase bars with the current stage named; forms use large touch targets and input modes (`inputMode="decimal"` on money fields). Client-rendered overlays portal to `<body>` to escape blurred-header containing blocks.

15. Performance & Scalability

Honest current limits and their upgrade paths — none requires a rewrite:

Concern	Today	Upgrade path
Rate limiting	In-memory (per-instance on serverless)	Redis/Upstash, same call signature <code>rateLimit</code>
File storage	Bytes in Postgres (simple, transactional)	S3/Supabase Storage + signed URLs before department-wide rollout <code>uploadFile</code>
Analytics	Computed on request	Nightly precompute + on-write invalidation before a lecturer views 40 groups <code>analytics</code>
Background jobs	None (all work inline)	Queue (e.g. pg-boss) unlocking scheduled reminders and automatic AI <code>queue</code>
Repository search	ILIKE + array facets	<code>tsvector</code> + GIN at $\sim 10^4$ records <code>search</code>
DB cold starts	Neon free tier suspends when idle	Always-on compute at production <code>compute</code>

Architecture: Next.js 15 App Router (server components + server actions as the entire backend), Prisma 6 / PostgreSQL, Auth.js v5. Deploys on Vercel + Neon. Modular `lib/` layer (policy, finance, lifecycle, activity, repository) keeps domain logic out of routes — the pure-function core (money, permissions, pacing, checklists) is what makes 200+ unit tests possible without a database.

16. Future SaaS & Multi-Tenancy

Target model: universities subscribe; each gets branding, departments, repository, administrators, analytics — securely isolated. Deliberately **not built now**; deliberately **not blocked** either. The prepared seams:

- Tenancy becomes a `University` entity above `Department` — one column and a scoping rule, because no query today assumes global visibility beyond what department scoping already constrains.
- `PublishedVisibility` already distinguishes department/university scope; a cross-university tier is a new enum value.
- Plans/quotas: a `Plan` enum and the `canUseAI` gating chain already exist; feature restriction points to the same chokepoints.
- The policy layer means tenant-isolation checks land in one function, not forty files.

MVP policy: every authenticated user gets the full feature set.

17. Phased Roadmap

Completed (each phase verified by build + tests before the next): foundation & auth → groups/departments → tasks/calendar/files/meetings → budget v1 → AI layer + kill switch → security hardening & account lifecycle → mobile passes → IA redesign (5-item nav, My Work, dashboard) → lifecycle & pacing → policy layer + capabilities + activity → finance v2 (verification ledger) → repository & publication → withdrawal workflow → policy consolidation → leader workspace.

Next, in order:

1. **Entity unification** — merge Workspace/Project routes into `/projects/[id]` with redirects; the `routes.ts` seam makes this one focused change. The largest remaining UX debt.
2. **Onboarding v2** — branching first-run (join vs create paths), building on the state-aware checklist; never modal-trapped, always resumable.
3. **Auth page consistency** — Register/Forgot/Reset styled to the login design system.
4. **Infrastructure tier** — Redis rate limiting, S3 file storage, job queue; then scheduled reminders and (if re-enabled) queued AI.
5. **Analytics precompute** — before real multi-group supervisor load.
6. **Multi-tenancy** — when a second institution is actually in the conversation.

Standing verification discipline: schema-touching changes ship with migration + `npm run build` + `npm test` green before the next phase; pure logic ships with unit tests in the same

commit; the owner's manual QA plan (§15-phase browser testing) remains to be executed against the current surface.

18. Decision Log — Rejected & Replaced

The most important section for future maintainers. These were considered seriously and declined *for reasons* — reopening them requires new evidence, not enthusiasm.

Per-student Engineering Score (0-100) → rejected. Gameable (message-count “communication” trains filler; activity metrics punish deep workers), unfair (penalises connectivity and work patterns, unevenly distributed in the Zimbabwean context), and false precision (a weighting in a helper function silently becomes an academic judgement). Replacement: raw signals shown side-by-side to staff; a composite exists only at **project** level (“at risk” is actionable and blames nobody).

AI plagiarism / “similarity” detection → rejected outright. An LLM has no corpus; it fabricates confident answers in both directions, and a false plagiarism flag is a disciplinary-grade harm. Real similarity requires a real indexed service.

AI screenshot-forgery detection → rejected. Vision models cannot reliably detect manipulation; a false positive accuses a named student of financial fraud, a false negative stamps unearned trust on a forgery. Replacement: DB-enforced reference uniqueness, receipt hashing, OCR cross-check warnings — deterministic, reproducible, explainable.

Score-gated publication (≥ 85) → rejected. Numeric gates get gamed and block legitimate work over formula quirks. Replacement: artifact checklist + supervisor signature.

Six role tiers → rejected. “Assistant Leader” is one capability, not an identity; each extra tier multiplies missable permission checks. Replacement: three roles + delegated capabilities, exhaustively tested.

Building real-time collaboration → rejected. Docs+Miro+GitHub is multi-year work; serverless can't hold the sockets; async-first serves intermittent connectivity better. Replacement: integrate external tools, make async excellent.

Payment processing → rejected permanently. Regulatory (money transmission), security, and trust grounds. The platform records and verifies; providers move money.

“AI everywhere,” auto-triggered → rejected. Free-tier quotas make it self-breaking (observed in production); replacement is the three-rules model (§13).

Three separate scoring systems (student score + analytics ranks + competitive badges) → collapsed. One health concept, attached to projects; any future achievements must be non-comparative.

This document is the long-term blueprint. Update it when decisions change — a spec that disagrees with the code is worse than no spec. The companion documents (SRS v1, Development Journal, Configuration Guide, User Manual, Maintenance Manual, Security Audit) live alongside it in `docs/`.